MDQ Specification Audit: mdq-spec

Date: 2026-10-09

Scope: Specification of the `numeric` and `ordering` question types: `docs/question-types/numeric.md`, `docs/question-types/ordering.md`, their schemas `schema/numeric.yaml` and `schema/ordering.yaml`, and the shared documents they depend on (`question-base.md`, `references/grammar.md`, `references/patterns.md`, `exam.md`, `responses.md`).

Methodology

The audit covers the numeric and ordering question types of the MDQ specification: their documents in `docs/question-types/`, their JSON schemas in `schema/`, and the shared documents they depend on (`question-base.md`, `references/grammar.md`, `responses.md`, `exam.md`, `lint-codes.md`). `scripts/spec_audit.py` passed all 14 steps before the audit started.

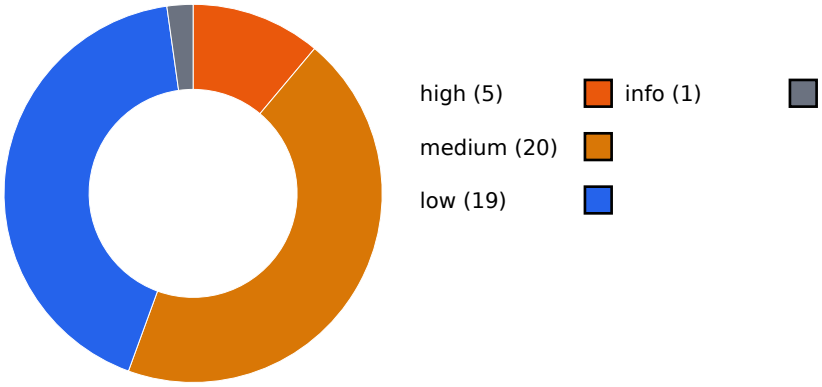
The lead auditor read every in-scope document and recorded contradictions, ambiguities, omissions, inconsistencies and schema mismatches. One subagent converted each Lark grammar into a runnable grammar with stub rules and tested what it accepts and rejects with Lark 1.3.1, and cross-checked the token regexes in Python `re`, JavaScript RegExp and JSON Schema validators. Two further subagents, one per question type and without the lead's notes, reviewed each type independently, compared it with its schema, checked the corpus under `examples/`, and ran every complete Markdown example through `mdq validate` (mdq-py).

Spelling and grammar mistakes were fixed in place without moving lines. Findings outside the two question types (short-answer grammar and pattern strings) were dropped as out of scope.

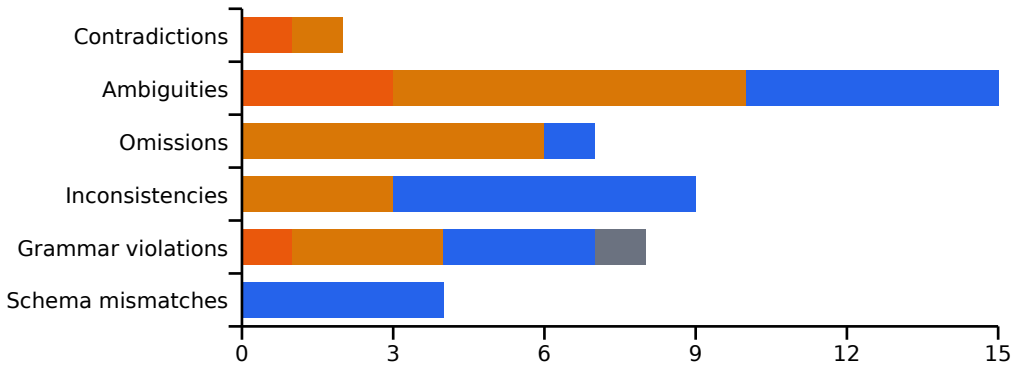
Executive summary

Severity	#
CRITICAL	0
HIGH	5
MEDIUM	20
LOW	19
INFO	1
Total findings	**45**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Grammar violations:** UNICODE_SPACE is exactly the Unicode White_Space property, and it matches the same 25 code points in Python re and in JavaScript RegExp with and without the u flag. Checked by matching every code point from U+0000 to U+10FFFF (surrogates skipped) and comparing with /\p{White_Space}/u in Node 22 (/tmp/grammar-audit/cls.py, cls.js). (docs/references/grammar.md:80-89)

✓ **Grammar violations:** The tokens ws, nl, SLUG, INTEGER, DECIMAL, UNIT and COMMENT_LINE accept the same strings in Python re and in JavaScript RegExp (with and without u) on 55 inputs: empty string, 0, 00, 007, -1, .5, 5., 1e3, Arabic-Indic and fullwidth digits, a-b, a--b, -a, a-, a_-b, Amazônia, space, tab, CRLF, CR, LF, LF CR, U+2028, U+0085, form feed, comments with CR, CRLF and an embedded U+2028, km/h, µm, °C, Ω, astral characters (U+1D707, emoji), U+00A0, U+200B, U+FEFF, U+001F, NUL and brackets. UNIT with + also matches astral characters in non-u mode, since it accepts both surrogate halves. INTEGER and DECIMAL reject signs, leading zeros, exponents and non-ASCII digits as grammar.md:69 states (/tmp/grammar-audit/tok.py, tok.js). (docs/references/grammar.md:58-75)

✓ **Grammar violations:** No Lark snippet in docs/ uses the shorthand classes \s, \d or \w, and the exam TAG terminal excludes the same 25 UNICODE_SPACE code points as grammar.md:83-85 states (checked by grep over every ```lark block of docs/question-types/ and docs/exam.md). (docs/exam.md:206)

✓ **Grammar violations:** The numeric grammar accepts every body example of numeric.md (42, 3.14, 3.14 +- 0.1, 3.14 +- 5%, 3.14 +- 5% +- 0.1, -1/3, [numeric(kg)]: 5.5 +- 0.1) plus +42, -0, 2.50, the reverse tolerance order, indentation, spaces around the colon, no spaces around +-, and the units µm, °C, km/h, Ω. It rejects empty input, an empty answer, 007, .5, 5., 1e3, '- 1', --1, 1/-3, 1.5/2, 1,5, 1_000, '1 000', negative and fraction tolerances, two absolute or two relative tolerances, ±, spaces inside the brackets, '[numeric (kg)]', empty and spaced units, units with parentheses, U+00A0 as whitespace, fullwidth and Arabic-Indic digits, a trailing newline and trailing text. No input produced an ambiguous parse with the dynamic or dynamic_complete lexer (Lark 1.3.1, /tmp/grammar-audit/test_numeric.py). The only defect found is the zero denominator. (docs/question-types/numeric.md:63-76)

✓ **Grammar violations:** The ordering observation rules enforce the stated rules: feedback and comment in either order are accepted; '> f1 ! c > f2', '! c > f ! c2' and two feedback blocks split by a blank line are rejected; '[ordering]', '##[accept]', prose between the block and a section, and a section without a content block are rejected (/tmp/grammar-audit/test_more.py). (docs/question-types/ordering.md:146-155)

✓ **Schema mismatches:** The numeric answer string pattern, the `unit` pattern and the `minimum: 0` constraints of the schema match the `sign? value` grammar, the `UNIT` terminal and the Additional Rules table. (schema/numeric.yaml:26-36, schema/numeric.yaml:45-47, schema/numeric.yaml:77-90)

✓ **Omissions:** Every numeric lint code of the Additional Rules table is exercised by the corpus, and the grading corpus covers rounding, both tolerances, negative and zero answers, and malformed responses. (examples/grading/numeric.yaml:17-185)

✓ **Omissions:** Both complete Markdown examples of ordering.md (the minimal Fibonacci question and the full example with strict indentation, extra, accept and reject) pass `mdq validate` with no warning. (docs/question-types/ordering.md:16-89)

✓ **Omissions:** Every ordering lint code of the Additional Rules table has a corpus example, including model-only examples for `accept-reject-overlap` and parse-error examples for type mismatch, two extras, interleaved observations, multi-line items and ordered lists. (docs/question-types/ordering.md:352-372)

✓ **Schema mismatches:** The enums and defaults of `indentation`, `unmatched` and `normalizations`, the `[level, text]` line shape with a non-negative integer level, `uniqueItems` on normalizations and `minItems: 2` on `lines` match the frontmatter footnotes and the Additional Rules table. (schema/ordering.yaml:17-156)

Central risks

• Numeric grading of boundary responses and of responses with units depends on the implementation: the spec does not fix the arithmetic of the tolerance test and gives two incompatible rules for units.

• The ordering rules on "the same lines" read literally as a multiset, which would make common valid questions fail with a critical `accept-reject-overlap`.

• Several grammars in scope reject documents that the prose and the corpus declare valid (indented `[ordering]` tag, blank line before a section), so an implementation that follows the grammar fails the shared test suite.

• The mapping from the Markdown body to the AST fields (`answer`, `tolerance`, `lines`, `extra`, `accept`, `reject`) exists only in the schemas and the corpus, so a new implementation must reverse-engineer it.

Detailed findings

Contradictions

Severity	Location	Description
HIGH	docs/responses.md:129-135	**F1** A response unit other than the declared one is both convertible and malformed
MEDIUM	docs/question-types/ordering.md: 181-183	**F2** Ordering puts the epilogue after its sections; the base puts special sections after the epilogue

F1: A response unit other than the declared one is both convertible and malformed

Location: docs/responses.md:129-135

```
Implementations MAY do unit conversions and rewrite the response to a canonical
form, but they MUST reject a response with a unit that is not compatible with
the question's unit with grade zero.

A value the question cannot represent is a malformed response, never a wrong
answer: a decimal where `domain` is `integer`, or a unit other than the
declared one.
```

Description: Line 129 lets an implementation convert a compatible unit (numeric.md "Unit conversion" gives 1kg for a g question). Line 133 says any unit other than the declared one is malformed. Line 130 also says an incompatible unit is both "rejected" and graded zero, while the Errors table says a response the question cannot represent raises `ResponseError`, never a score.

Impact: For `[numeric(g)]: 1000` and the response `{value: 1, unit: kg}`, one implementation scores 1, another raises `ResponseError`. For `{value: 1, unit: s}`, one scores 0 and another raises an error.

Suggested fix: State one rule: a unit equal to the declared one is valid; a different unit that the implementation can convert is converted before grading; any other unit is a malformed response (`ResponseError`), not a zero. Remove "with grade zero" and "or a unit other than the declared one".

F2: Ordering puts the epilogue after its sections; the base puts special sections after the epilogue

Location: docs/question-types/ordering.md:181-183

```
The sections below follow the content block directly. The epilogue, if any,
comes after the last section; prose between the block and a section is an
error (`parse-error`).
```

Description: question-base.md:4-6 and :230-231 fix the order frontmatter, introduction, body, epilogue, then the question-specific sections. Ordering requires the `## [extra]`, `## [accept]` and `## [reject]` sections right after the body and the epilogue after them. The ordering grammar (ordering.md:141) has no epilogue at all.

Impact: An author following the base document writes the epilogue before `## [accept]` and gets a `parse-error`. Readers cannot tell which document is normative.

Suggested fix: Make question-base.md describe the type-specific sections as part of the body (or as coming before the epilogue), and add the epilogue to the ordering grammar.

Ambiguities

Severity	Location	Description
HIGH	docs/question-types/ordering.md:208-212	**F3** "Hold the same lines" can mean the same multiset or the same sequence
HIGH	docs/question-types/ordering.md:261-266	**F4** Indentation of `ul` lines is not defined
HIGH	docs/question-types/numeric.md:148-154	**F8** Tolerance comparison arithmetic is not specified
MEDIUM	docs/question-types/ordering.md:106-108	**F6** Declared `content` that disagrees with the body block has no meaning
MEDIUM	docs/question-types/ordering.md:326-332	**F7** Ordering score is "always binary" but a pending response is scored by hand
MEDIUM	docs/question-types/numeric.md:166-169	**F9** Inferred `decimalPlaces` of 0 turns a declared `decimal` domain into rounding to integers
MEDIUM	docs/responses.md:102-111	**F12** Numeric response forms are underspecified
MEDIUM	docs/question-types/numeric.md:114-115, docs/question-types/numeric.md:166-168, docs/question-types/numeric.md:101-102	**F29** Domain and decimal-place inference from a JSON/YAML number is undefined
MEDIUM	docs/question-types/ordering.md:296-298	**F39** Feedback is undefined when a response matches several sections
MEDIUM	docs/question-types/ordering.md:261-272	**F42** Indentation step offered to the student under `strict` is not defined
LOW	docs/question-types/ordering.md:234-237	**F17** Indented blockquotes in observations are implementation-defined
LOW	docs/question-types/numeric.md:215	**F35** An empty `tolerance` object is neither forbidden nor covered by `missing-tolerance`
LOW	docs/question-types/ordering.md:165-166	**F44** Whitespace and list markers in line text are not specified
LOW	docs/question-types/ordering.md:193-194	**F45** Language of `## [accept]` and `## [reject]` fences is not addressed

Severity	Location	Description
LOW	docs/question-types/ordering.md: 7-9	**F47** "Never shown that order" conflicts with a random shuffle

F3: "Hold the same lines" can mean the same multiset or the same sequence

Location: docs/question-types/ordering.md:208-212

```
A question SHOULD NOT declare two sections whose content
blocks hold the same lines once **all forms of normalization** have been
applied
```

Description: Every alternative of an ordering question is usually a permutation of the same lines. Read as "the same set of lines", the MUST NOT forbids almost every question that has both an accepted and a rejected ordering (a reject is typically the correct lines in a wrong order). The intended reading is "the same lines in the same order". The same wording is used by `accept-reject-overlap`, `duplicate-alternative-lines` and `accept-repeats-answer-key` (ordering.md:358, :366-367, lint-codes.md:13, :24, :75).

Impact: An implementation that reads it as a multiset raises a critical `accept-reject-overlap` on valid questions.

Suggested fix: Write "the same sequence of lines (same lines, same order)" in the prose, in the three Additional Rules rows and in lint-codes.md.

F4: Indentation of `ul` lines is not defined

Location: docs/question-types/ordering.md:261-266

```
The **indentation unit** is inferred from the lines of the question. It
considers all lines in the `[ordering]`, `## [extra]`, `## [accept]` and
`## [reject]` blocks, normalizes a tabulation as 4 spaces, and takes the
greatest common divisor of the leading whitespace of every non-blank line that
is indented at all.
```

Description: For a code block, "leading whitespace" is clear. For a `ul` list it is not: the spaces before the `-` marker, list nesting, or spaces after the marker. A nested list item is also a continuation of its parent in CommonMark, and ordering.md:167-168 makes an item with continuation lines a `parse-error`. So it is unclear whether `text` content can have levels other than 0. Also unspecified: whether a tab counts as 4 spaces or advances to the next tab stop ("`\t`" is 6 or 4), and what level a whitespace-only line has.

Impact: Two implementations give different `[level, text]` pairs for the same `ul` question, so the same response grades differently.

Suggested fix: State that `text` content is always at level 0 (nested lists are a `parse-error`), or define levels from list nesting. Define tab expansion and the level of a blank line.

F8: Tolerance comparison arithmetic is not specified

Location: docs/question-types/numeric.md:148-154

```
* Absolute tolerance: The response is accepted if `|answer - response| ≤ abstol`.
```

Description: Only rounding is required to use decimal arithmetic (numeric.md:160-161). The subtraction and the ≤ test are not. In binary floating point, `[numeric]: 3.14 +- 0.1` with the response `3.24` gives `|3.14 - 3.24| = 0.10000000000000009 > 0.1`, so a response exactly on the boundary is rejected; in decimal arithmetic it is accepted. The same applies to the relative tolerance and to a numeric response written as a float (`2.675` as a JSON number is `2.67499999...`).

Impact: Responses on the tolerance boundary grade differently across implementations and languages.

Suggested fix: Require the whole tolerance test to run in exact decimal/rational arithmetic, with a JSON number read as its shortest decimal representation.

F6: Declared `content` that disagrees with the body block has no meaning

Location: docs/question-types/ordering.md:106-108

```
[^1]: `content = "code" | "text"`. Both fields are OPTIONAL, and when given
they override what the body implies.
```

Description: `content: text` over a fenced code block, or `content: code` over a `ul`, "overrides" the body, but the spec does not say what that does: whether lines are still split per code line or per list item, whether `highlight` then applies, or whether it is an error. "Both fields" also refers to `content` and `highlight` from a footnote attached only to `content`. The schema adds `default: text` (schema/ordering.yaml:60), which the prose never mentions.

Impact: Implementations can accept, reject, or reinterpret the same document.

Suggested fix: Make a mismatch between declared `content` and the body block a `parse-error` (or a warning with a defined effect). Document the `text` default for documents written as YAML/JSON.

F7: Ordering score is "always binary" but a pending response is scored by hand

Location: docs/question-types/ordering.md:326-332

```
The score is always binary -- `1` or `0`, with no partial credit for a nearly
correct ordering.
```

Description: Under `unmatched: manual` the instructor supplies the score of a pending response. The spec does not say whether that score must also be 0 or 1, or can be any value in [0, 1] (the essay case).

Impact: A host may refuse a partial manual score that another host accepts.

Suggested fix: State that the binary rule applies to automatic grading only, and that a manual score is any value in [0, 1] (or restrict it explicitly).

F9: Inferred `decimalPlaces` of 0 turns a declared `decimal` domain into rounding to integers

Location: docs/question-types/numeric.md:166-169

```
When the frontmatter does not give `decimalPlaces` and the domain is
`decimal`, it is inferred from the body: the larger of the number of decimal
places written in the value and in the absolute tolerance.
```

Description: With `domain: decimal` declared and `[numeric]: 42` (or `1/3`) and no absolute tolerance, the rule gives `decimalPlaces: 0`. Both sides are then rounded to integers: `41.6` rounds to 42 and is accepted, and for `1/3` every response in [-0.5, 0.5) is correct. The spec does not say whether this is intended, and no lint reports it.

Impact: A question meant to be strict silently accepts a wide range.

Suggested fix: Specify the result for a value with no written decimal places (for a fraction, an exact comparison or a lint), or add a lint when the inferred `decimalPlaces` is 0 under a declared `decimal` domain.

F12: Numeric response forms are underspecified

Location: docs/responses.md:102-111

```
A string carries an exact rational, which no float can represent. It MUST be
used when the question's `domain` is `fraction` and the value is not exactly
representable as a decimal:
```

Description: Unspecified: whether a `fraction` question accepts a decimal response (`0.5` for `1/2`) and how a decimal response that approximates a fraction (`0.333`) is graded; whether `"4/2"` or `"2/4"` is valid where an integer or a reduced fraction is expected; whether a decimal string may be an integer (`"3"`) and what grammar it follows. Line 111 is garbled ("which systems MAY flag the system to use decimal arithmetic"). The table at line 30 lists only a number or a rational string, and omits the decimal string and the `{value, unit}` struct. Line 125-127 says a response unit is malformed when the question has no unit, but does not say whether a unitless response to a question with a unit is valid.

Impact: Implementations disagree on which responses are malformed and which are graded.

Suggested fix: Specify the response grammar per domain (reuse `sign? value`), the treatment of unreduced fractions and unitless responses, and update the table at line 30.

F29: Domain and decimal-place inference from a JSON/YAML number is undefined

Location: docs/question-types/numeric.md:114-115, docs/question-types/numeric.md:166-168, docs/question-types/numeric.md:101-102

```
If not given in the frontmatter, the domain is inferred from how the value and
the absolute tolerance are written
```

Description: Inference reads how numbers are written (`8 +- 1.0` is `decimal` with 1 place, examples/valid/numeric/whole-decimal-tolerance). A document written directly as YAML/JSON may omit `domain` and `decimalPlaces` (the grading corpus does), but a YAML or JSON number does not keep its spelling: `1.0` and `1`, `2.50` and `2.5` are the same value. mdq-py infers `integer` for `{answer: 8, tolerance: {absolute: 1.0}}` and rejects the response `8.5` as malformed, while the Markdown `[numeric]: 8 +- 1.0` gives a `decimal` question that accepts it. The answer `2.50` (number) infers 1 place, the string `"2.50"` 2 places, which contradicts line 101-102 (the domain is unaffected by the representation).

Impact: The same question grades differently depending on whether it was written in Markdown or as data, and across YAML/JSON libraries.

Suggested fix: State that for data documents inference reads values, not spelling (a whole number is an integer, decimal places of the shortest representation), or require `domain` and `decimalPlaces` in data documents.

F39: Feedback is undefined when a response matches several sections

Location: docs/question-types/ordering.md:296-298

```
A response that matches an accepted or a rejected answer shows that section's
feedback, if it declares one.
```

Description: Two `## [accept]` sections with the same lines (`duplicate-alternative-lines`) and an accept that repeats the `[ordering]` block (`accept-repeats-answer-key`) are warnings, not errors, so a valid question can have several sections that match one response, each with its own feedback. The spec does not say which feedback is shown: the first, all, or none (the answer key has no feedback).

Impact: Implementations show different feedback for the same response.

Suggested fix: State a precedence rule, for example: the `[ordering]` block first, then sections in document order; the first match decides the score and the feedback.

F42: Indentation step offered to the student under `strict` is not defined

Location: docs/question-types/ordering.md:261-272

```
A line's **indentation level** is its leading whitespace divided by that unit,
and that level -- not the raw whitespace -- is what a response carries and
what `"strict"` compares.
```

Description: The unit is the GCD over every section, so a single distractor or accepted line indented by 2 spaces moves 4-space answer-key lines to levels 2 and 4 (examples/valid/ordering/indentation-unit-across-sections.yaml). The spec does not say what one indentation step in the student interface is (one unit, or a code-level indent), nor that adding a section can change the levels of the answer key and of stored responses.

Impact: Under `strict`, a student who indents a loop body once submits level 1 while the key expects level 2; responses stored before a section was added stop matching.

Suggested fix: State that the interface moves a line by one unit per step and renders levels with the inferred unit, or derive the unit from the `[ordering]` block alone and lint other sections that do not fit it.

F17: Indented blockquotes in observations are implementation-defined

Location: docs/question-types/ordering.md:234-237

```
It is up to the implementation to decide if it should ignore the indentation,
issue a warning or even an error.
```

Description: The grammar (ordering.md:153-154) accepts `ws?` before `>` and `!`, while the prose lets implementations reject it. Indentation of 4 spaces or more also makes an indented code block in CommonMark, as the example shows. The same document can parse or fail depending on the implementation.

Impact: Portability of documents between implementations.

Suggested fix: Fix one behavior: accept up to 3 spaces of indentation, as CommonMark does, and lint it.

F35: An empty `tolerance` object is neither forbidden nor covered by `missing-tolerance`

Location: docs/question-types/numeric.md:215

```
| tolerance          | info     | missing-tolerance              | should be defined for a
`decimal` or `fraction` answer[^5]         |
```

Description: schema/numeric.yaml:68-90 allows `tolerance: {}`. The rule says the tolerance "should be defined"; lint-codes.md:37 says "No `tolerance` is given". mdq-py reports nothing for `{answer: 2.5, tolerance: {}}`, while the same question without the key reports `missing-tolerance`.

Impact: An empty tolerance hides the lint without adding any tolerance.

Suggested fix: Require at least one property in the schema (`minProperties: 1`), or state that an empty tolerance counts as missing.

F44: Whitespace and list markers in line text are not specified

Location: docs/question-types/ordering.md:165-166

```
If it is a `ul` list, the question infers `content: "text"`. Each line is
derived from the respective list item, preserving its markdown source verbatim.
```

Description: "Markdown source verbatim" does not say whether the list marker and the space after it are part of the text (they are not, in the corpus), or whether trailing whitespace is kept. For code lines, the spec says nothing about trailing whitespace either, and it is compared by content.

Impact: A line with a trailing space can be different from the same line without it in one implementation and equal in another.

Suggested fix: State that the text excludes the marker and its following whitespace, and whether trailing whitespace is stripped for both content types.

F45: Language of `## [accept]` and `## [reject]` fences is not addressed

Location: docs/question-types/ordering.md:193-194

```
The highlight of fenced code blocks is ignored.
```

Description: The sentence sits in the `## [extra]` section. The accepted/rejected section only says the content block must have the same type. Nothing says whether a fence language in `## [accept]` or `## [reject]` is ignored, must match `highlight`, or is an error.

Impact: A different fence language in an alternative can be accepted, warned or rejected depending on the implementation.

Suggested fix: Move the sentence to the Body section and make it apply to every section after `[ordering]`.

F47: "Never shown that order" conflicts with a random shuffle

Location: docs/question-types/ordering.md:7-9

```
Students are never shown that order --
implementations MUST present the lines shuffled, in a random or pseudo-random
arrangement
```

Description: A random permutation can equal the answer key, especially for two or three lines (1 in 2 for the minimum of two lines). The text says both "never" and "random" without saying whether an implementation must reshuffle in that case.

Impact: An implementation that follows "random" literally can show the solution.

Suggested fix: Say that implementations MUST NOT present the answer key's order, and reshuffle when the permutation equals it.

Omissions

Severity	Location	Description
MEDIUM	docs/question-types/ordering.md: 203-212	**F5** No rule ties accepted and rejected orderings to the lines shown to the student
MEDIUM	docs/question-types/numeric.md: 112-117	**F28** The spec never states what the domain does to responses
MEDIUM	docs/question-types/numeric.md: 22	**F30** A frontmatter `unit` that disagrees with the tag unit is silently dropped
MEDIUM	docs/question-types/numeric.md: 145-150	**F31** Mapping from the body to `answer` and `tolerance` is not specified
MEDIUM	docs/question-types/ordering.md: 133-136, docs/question-types/ordering.md:354-358	**F40** Mapping from the ordering body to `lines`, `extra`, `accept` and `reject` is not specified
MEDIUM	docs/question-types/ordering.md: 308-332	**F41** No grading examples for ordering in the corpus
LOW	docs/responses.md:187-190	**F46** A response that changes a level under `fixed` has no defined outcome

F5: No rule ties accepted and rejected orderings to the lines shown to the student

Location: docs/question-types/ordering.md:203-212

```
The accept/reject blocks start with a `## [accept]` or `## [reject]` header,
followed by optional observations and the content block.
```

Description: Nothing says an accepted or rejected ordering must use only lines from `[ordering]` plus `## [extra]`, nor how many copies. The schema calls an alternative "One accepted or rejected ordering of the question's lines" (schema/ordering.yaml:162), which implies the constraint. Also unspecified: whether an `## [accept]` may contain a distractor (ordering.md:197 says distractors MUST NOT appear in a correct response), and what happens when a `## [reject]` equals the `[ordering]` block after normalization (`accept-reject-overlap` covers only accept vs reject). Response-side validation is covered by a separate finding.

Impact: A typo in an alternative makes it unreachable without any diagnostic. A reject that equals the answer key has no defined outcome.

Suggested fix: Add rules: alternative lines must come from the presented pool (lint, warning); a reject must not equal the answer key (critical).

F28: The spec never states what the domain does to responses

Location: docs/question-types/numeric.md:112-117

```
If not given in the frontmatter, the domain is inferred from how the value and
the absolute tolerance are written
```

Description: "Number type/domain" explains how the domain is inferred, but numeric.md never says what the domain means for grading. The effect lives only in responses.md:133-134 (a decimal response to an `integer` question is malformed) and in examples/grading/numeric.yaml ("Exact integer": `1822.5` is malformed; "Fraction": `0.3333` scores 0 for `1/3`, so a `fraction` question accepts decimals). Grading (numeric.md:189-199) and Tolerances (numeric.md:143-154) do not mention malformed responses. The schema describes `domain` only as "The domain/kind of number for the response".

Impact: A reader of numeric.md cannot tell that `[numeric]: 42 +- 5%` rejects the response `43.5` as malformed even though it is inside the tolerance.

Suggested fix: Add to "Number type/domain" the set of responses each domain accepts, and link the malformed-response rule of responses.md.

F30: A frontmatter `unit` that disagrees with the tag unit is silently dropped

Location: docs/question-types/numeric.md:22

```
| unit          | string    | Unit of the response, if given                  |
```

Description: numeric.md does not say what happens when the frontmatter `unit` and the `[numeric(unit)]` tag disagree. The generic rule of question-base.md:109-110 (frontmatter wins) applies, and examples/valid/numeric/frontmatter-unit-wins accepts `unit: kg` with `[numeric(g)]: 60 +- 1` with no diagnostic. short-answer.md:95 makes the analogous body/frontmatter collision an error (`conflicting-accept`).

Impact: A question whose body says grams is graded in kilograms, with no warning; numeric and short-answer treat collisions differently.

Suggested fix: Specify the collision in numeric.md and report it, at least as a warning, when the two units differ.

F31: Mapping from the body to `answer` and `tolerance` is not specified

Location: docs/question-types/numeric.md:145-150

```
* Relative tolerance: The response is accepted if `|answer - response| ≤ tol`,
```

Description: numeric.md never declares the `answer` and `tolerance` fields nor says that the body value becomes `answer`, `+- x` becomes `tolerance.absolute` and `+- x%` becomes `tolerance.relative` divided by 100. The frontmatter table (line 19-24) lists neither, and mdq-py reports both as `unknown-frontmatter-key` in the frontmatter. Line 150 uses the body percentage (`÷ 100`), footnote 2 (line 217) uses the stored fraction (`|answer| ≤ relative`); only the comment of examples/valid/numeric/tolerances.yaml states that 5% becomes 0.05.

Impact: Implementers must reverse-engineer the data form from the schema and the corpus; a `relative: 5` written for 5% is a likely error.

Suggested fix: Add a section that maps each body part to its field, with `tolerance.relative = percentage / 100`, and use one form in the formulas.

F40: Mapping from the ordering body to `lines`, `extra`, `accept` and `reject` is not specified

Location: docs/question-types/ordering.md:133-136, docs/question-types/ordering.md:354-358

```
| lines                 | critical | schema-error                | must have at least two
lines (schema)                          |
```

Description: The prose talks only about the `[ordering]` block and the `## [extra]`, `## [accept]` and `## [reject]` sections. It never says that they become the fields `lines`, `extra`, `accept` and `reject`, that each alternative is an object with `lines`, `feedback` and `comment`, or how a `>` block and a `!` block turn into those strings (joined lines, `>` stripped). The Additional Rules table then uses those field names without introducing them.

Impact: Readers must reverse-engineer the YAML/JSON form from the schema and the corpus; the feedback and comment text can be serialized differently by two implementations.

Suggested fix: Add a short paragraph or a YAML example that shows the parsed form of the full example, and state how feedback and comment text is extracted.

F41: No grading examples for ordering in the corpus

Location: docs/question-types/ordering.md:308-332

```
Grading compares the response to the answer keys **by content**: lines carry no
```

Description: examples/grading/ has files for multiple-choice, multiple-selection, numeric, short-answer and true-false, but none for ordering. No example exercises grading by content, duplicated lines, dedent, `skip-blanks`, `fixed`/`lenient`/`strict`, `unmatched`, a distractor in the response, or the per-section feedback.

Impact: The grading rules of ordering, the most complex among the automatically graded types, are not checked by the shared test suite, so the Python and TypeScript implementations can diverge.

Suggested fix: Add examples/grading/ordering.yaml covering each normalization, each indentation mode, duplicates, distractors, and both `unmatched` values.

F46: A response that changes a level under `fixed` has no defined outcome

Location: docs/responses.md:187-190

```
The indentation level is always present. Under `indentation: fixed` it is the
authored level, which the student cannot change
```

Description: Under `fixed` the student cannot change a level, but a response is data and can carry any level. The spec does not say whether a response with a level other than the authored one is malformed or simply a wrong answer. The same gap exists for a line that is in no section at all, and for a line used more times than the question provides; responses.md:184-185 covers only distractors.

Impact: One implementation rejects the response as malformed, another grades it 0 or leaves it pending.

Suggested fix: State that a response line whose text and level are not in the presented pool, or that is used more often than provided, is a malformed response (`ResponseError`).

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/numeric.md:226-227, docs/question-types/numeric.md:42-43	**F10** `missing-tolerance` says only an exact match is accepted, but decimals are rounded first
MEDIUM	docs/question-types/numeric.md:207-211	**F11** `answer-outside-domain` and `domain-mismatch` report the same condition at two severities
MEDIUM	docs/question-types/ordering.md:118-120, docs/question-types/short-answer.md:266-267	**F37** `unmatched` defaults differ between ordering and short-answer
LOW	docs/question-types/numeric.md:38-41	**F15** Numeric body examples are written as H1 headings in a Markdown block
LOW	docs/question-types/numeric.md:97-99	**F16** Integer coercion limit is 32-bit while the precision rule allows 53-bit
LOW	docs/question-types/numeric.md:127-128, docs/question-types/numeric.md:97	**F32** Representation rules cover the answer but not the absolute tolerance
LOW	docs/question-types/numeric.md:5-6	**F33** Introduction says numeric questions declare a tolerance, which is optional
LOW	docs/question-types/numeric.md:210	**F34** `relative-tolerance-over-one` blames a units mistake that a Markdown percentage cannot make

Severity	Location	Description
LOW	docs/question-types/ordering.md: 100-101, docs/question-types/essay.md:21-22	**F38** Ordering `content` and essay `input` declare the same code/text choice under different names

F10: `missing-tolerance` says only an exact match is accepted, but decimals are rounded first

Location: docs/question-types/numeric.md:226-227, docs/question-types/numeric.md:42-43

```
[^5]: An exact comparison of a non-integer response is rarely what the author
means, since the number of digits the student types decides the outcome.
```

Description: For a `decimal` question, the response is rounded to `decimalPlaces` before the test (numeric.md:158-162), so `[numeric]: 3.14` accepts `3.141`, and extra digits never decide the outcome. lint-codes.md:37 repeats "only an exact match is accepted". For a `fraction` question, the student types an exact rational, so the rationale does not apply either.

Impact: The lint rationale misleads authors about what the question accepts.

Suggested fix: Reword the footnote and lint-codes.md: without a tolerance, a decimal response must round to the answer at `decimalPlaces`; reconsider whether the lint applies to `fraction`.

F11: `answer-outside-domain` and `domain-mismatch` report the same condition at two severities

Location: docs/question-types/numeric.md:207-211

```
| answer             | warning  | answer-outside-domain          | must be a whole number when
`domain` is "integer"[^6]              |
```

Description: Footnote 3 defines `domain-mismatch` (info) as `integer` or `fraction` with a number that is not whole, or `integer` with a fraction string. `answer-outside-domain` (warning) covers `integer` with a non-whole answer. `domain: integer` with `answer: 3.5` triggers both. The `domain-mismatch` row says "narrower than the values", footnote 3 talks only of the answer, and lint-codes.md:23 says "the answer and the absolute tolerance": it is unclear whether `domain: integer` with `+- 0.5` is a mismatch.

Impact: Duplicate diagnostics at different severities; implementations disagree on whether the absolute tolerance counts.

Suggested fix: Give each condition one code. State in footnote 3 whether the absolute tolerance takes part, matching lint-codes.md.

F37: `unmatched` defaults differ between ordering and short-answer

Location: docs/question-types/ordering.md:118-120, docs/question-types/short-answer.md:266-267

```
[^4]: `Unmatched = "manual" | "incorrect"`. `"manual"` (the default) leaves a
```

Description: Both types declare a field `unmatched` with the same values and meaning. Short-answer defaults it to "incorrect" when the question has an `accept` pattern and to "manual" only when it has none. An ordering question always has an answer key (the `[ordering]` block), yet its default is always "manual". The same property, with the same rationale, gets opposite defaults.

Impact: An author who learns `unmatched` on short-answer expects unmatched ordering responses to score 0 by default; instead they stay pending and need manual grading.

Suggested fix: Use one default rule for `unmatched` across types, or state in ordering.md why its default differs from short-answer.

F15: Numeric body examples are written as H1 headings in a Markdown block

Location: docs/question-types/numeric.md:38-41

````
```md
# Simple integer
[numeric]: 42
````

Description: The labels are # lines in an `md` block, which are H1 headings in Markdown. H1 headings are forbidden elements in a question (question-base.md:134). A reader can copy them as part of a valid body.

Impact: Minor confusion; the block looks like a document but is not one.

Suggested fix: Use a table of examples or one block per example with the label in prose.

F16: Integer coercion limit is 32-bit while the precision rule allows 53-bit

Location: docs/question-types/numeric.md:97-99

```
It SHOULD be kept as a string if it is not representable in the 32-bit signed
integer range.
```

Description: numeric.md:90-91 defines loss of precision against the nearest float. Every integer up to 2^53 converts to a float without loss, yet integers above 2^31 must stay strings. The limit is not explained.

Impact: Surprising string answers for values like `390000000000`.

Suggested fix: Explain the 32-bit choice (integer type in some hosts), or align it with the float rule.

F32: Representation rules cover the answer but not the absolute tolerance

Location: docs/question-types/numeric.md:127-128, docs/question-types/numeric.md:97

```
The absolute tolerance is an `INTEGER` or a `DECIMAL`, never a fraction (see
the grammar above and `tolerance.absolute` in the schema, a number).
```

Description: The answer keeps a string form to avoid loss of precision (line 87-88) and an integer is coerced to an integral value (line 97). `tolerance.absolute` is number-only in schema/numeric.yaml:77-78, and the corpus writes `+- 1` as `absolute: 1.0` (examples/valid/numeric/decimal-integer-tolerance.yaml, frontmatter-unit-wins.yaml). So `+- 0.10` loses its written digits and a long tolerance loses precision, against the rule of line 87-88.

Impact: The data form of a tolerance does not round-trip its written precision, which feeds the inference ambiguity of data documents.

Suggested fix: Apply the answer representation rules to `tolerance.absolute` (allow the decimal string form), or state that tolerances are always floats and why.

F33: Introduction says numeric questions declare a tolerance, which is optional

Location: docs/question-types/numeric.md:5-6

```
Numeric questions expect a single numeric answer. They declare both the correct
answer and tolerance.
```

Description: The example right below (`[numeric(g)]: 1000`) has no tolerance, and the grammar makes `tolerances?` optional.

Impact: Readers may think a tolerance is required.

Suggested fix: Write: "They declare the correct answer and an optional tolerance."
```

F34: `relative-tolerance-over-one` blames a units mistake that a Markdown percentage cannot make

Location: docs/question-types/numeric.md:210

```
| tolerance.relative | info     | relative-tolerance-over-one    | is a fraction, so a value
above 1 is likely a mistake[^4]          |
```

Description: The rationale (footnote 4 and lint-codes.md:50) is that a fraction was written as a percentage. In Markdown the author writes the percentage itself: `[numeric]: 10 +- 150%` makes mdq-py report "it is written as a fraction, not a percentage", which is false for that source.

Impact: Misleading message for Markdown documents.

Suggested fix: Reword the rule as "a relative tolerance above 100% is likely a mistake", or restrict the units-mistake message to data documents.

F38: Ordering `content` and essay `input` declare the same code/text choice under different names

Location: docs/question-types/ordering.md:100-101, docs/question-types/essay.md:21-22

```
| content        | string          | Either "code" or "text". Inferred from the body when
omitted[^1].           |
```

Description: Both fields select between "code" and "text" and both pair with `highlight`, which is ignored unless the value is "code" (same `ignored-highlight` lint). Essay calls the field `input`, ordering calls it `content`. Essay also has the lint `code-input-without-highlight`; ordering has no equivalent for a code question with no `highlight`.

Impact: Authors must remember two names for one concept, and a code ordering question without a language gets no notice while an essay does.

Suggested fix: Use one field name for both types, or document the difference; add the missing-highlight info lint to ordering or remove it from essay.

Grammar violations

Severity	Location	Description
HIGH	docs/question-types/ordering.md:141	**F18** Ordering tag rule rejects an indented tag
MEDIUM	docs/question-types/ordering.md:141	**F20** Ordering grammar rejects a blank line between a content block and the next section
MEDIUM	docs/references/grammar.md:29, docs/references/grammar.md:63	**F22** nl and COMMENT_LINE can split CRLF into two line endings
MEDIUM	docs/references/grammar.md:17-19, schema/mdq.schema.json:396, schema/mdq.schema.json:483, schema/mdq.schema.json:490	**F26** Schema forms of SLUG, UNIT and the numeric answer accept a trailing newline in Python
LOW	docs/question-types/ordering.md:144	**F21** Ordering grammar accepts more than one extra section
LOW	docs/question-types/question-base.md:41	**F23** Frontmatter grammar does not force the first comment paragraph into comment_string
LOW	docs/question-types/numeric.md:75	**F25** Numeric fraction rule accepts a zero denominator
INFO	docs/question-types/ordering.md:150-151	**F24** Observations rule is ambiguous on blank lines after the first block

F18: Ordering tag rule rejects an indented tag

Location: docs/question-types/ordering.md:141

```
ordering : "[" "ordering" "]" nl content section*
```

Description: question-base.md:189 says a tag may be indented. The numeric `answer` rule starts with `ws?`, but the `ordering` rule does not, so `[ordering]` followed by a code block is rejected by the grammar. Tested with Lark 1.3.1 (Earley, ambiguity=explicit, stubs in /tmp/grammar-audit/). The same defect exists in the short-answer and essay tag rules (short-answer.md:150-151, essay.md:35), outside the scope of this audit.

Impact: The grammar rejects a document that the prose and the corpus declare valid. An implementation that follows the grammar fails the shared test suite; one that follows the corpus has no grammar for what indentation it may accept.

Suggested fix: Start `ordering` with `ws?`, as `answer` of numeric.md does, and do the same for the short-answer and essay rules.

F20: Ordering grammar rejects a blank line between a content block and the next section

Location: docs/question-types/ordering.md:141

```
ordering : "[" "ordering" "]" nl content section*
```

Description: A section header (`"##" ws "[" ...`) must follow `content` directly, with no `nl` between them, while the header is followed by `nl+`. The valid example examples/valid/ordering/accept-alternative.mdq.md has a blank line between the closing fence and `## [accept]`. With stubs where `md_code_block` and `md_ul` end with the line ending of their last line (/tmp/grammar-audit/ordering.lark): `'[ordering]\n- a\n- b\n\n## [accept]\n\n- b\n- a\n'` reject; the same input without the blank line before ## is accepted.

Impact: The grammar accepts only the layout without a blank line, which is the less common one, and contradicts the corpus.

Suggested fix: Allow blank lines before each section: `ordering : "[" "ordering" "]" nl content (nl* section)*`, or state in the prose that `md_code_block` and `md_ul` include the blank lines after them.

F22: nl and COMMENT_LINE can split CRLF into two line endings

Location: docs/references/grammar.md:29, docs/references/grammar.md:63

```
nl : /\r\n|\r|\n/
```

Description: grammar.md:32-33 says `nl` matches the three line endings of CommonMark, where CR is a line ending only when LF does not follow it. The regex also matches a lone `\r` in front of `\n`, so CRLF has two derivations. Test with Lark 1.3.1 (lexer=dynamic_complete, ambiguity=explicit, /tmp/grammar-audit/amb.py): `start: nl+` on `'\r\n'` gives two trees (one `nl`, two `nl`). In the frontmatter rule, `'---\r\n# c1\r\n# c2\r\nid: a\r\n---\r\n'` gives 12 ambiguities, among them a parse where `COMMENT_LINE` is `# c1\r`, the `\n` starts the YAML, and `# c2` is a YAML comment, so the comment string loses its second line.

Impact: A regex engine or a parser that backtracks into the alternation can count a CRLF as a line plus a blank line. Blank lines end the comment string and are significant in several bodies, so the meaning of a Windows file can change.

Suggested fix: Use `/\r\n|\r(?!\n)|\n/` for `nl` and `(\r\n|\r(?!\n)|\n)` at the end of `COMMENT_LINE`.

F26: Schema forms of SLUG, UNIT and the numeric answer accept a trailing newline in Python

Location: docs/references/grammar.md:17-19, schema/mdq.schema.json:396, schema/mdq.schema.json:483, schema/mdq.schema.json:490

```
"pattern": "^[a-zA-Z0-9]+(?:[-_][a-zA-Z0-9]+)*$",
```

Description: grammar.md:17-19 says each class matches the same characters in Python `re`, JavaScript RegExp and a JSON Schema `pattern`. The classes do, but the schema wraps them in `^...$`, and Python `$` also matches before a final `\n`. Test with jsonschema (Python) and Node 22: `'Q1\n'` against the SLUG pattern, `'kg\n'` against the unit pattern and `'42\n'` against the numeric answer pattern are valid in Python and invalid in JavaScript (with and without the u flag). The same `$` ends every other anchored pattern of the schema.

Impact: The two implementations validate the same YAML or JSON document differently: an id, a unit or an answer with a trailing line feed (easy to get from a YAML block scalar) passes the Python validator and fails the TypeScript one.

Suggested fix: End the patterns with `(?![\s\S])` or `$(?![\s\S])` instead of `$`, or have the Python validator use `re.fullmatch` semantics, and state the anchoring rule in grammar.md.

F21: Ordering grammar accepts more than one extra section

Location: docs/question-types/ordering.md:144

```
section : ordering_extra | ordering_accept | ordering_reject
```

Description: ordering.md:192 says there can be at most one `## [extra]` section. `section*` allows any number. Test (/tmp/grammar-audit/ordering.lark): `'[ordering]\n- a\n- b\n## [extra]\n- c\n## [extra]\n- d\n'` accept.

Impact: The grammar accepts a document the prose forbids. The prose is clear, so the effect is small.

Suggested fix: Express the limit in the grammar, for example `ordering : ... nl content (accept_or_reject)* (ordering_extra (accept_or_reject)*)?`, or add a note under the snippet that the limit is not in the grammar.

F23: Frontmatter grammar does not force the first comment paragraph into comment_string

Location: docs/question-types/question-base.md:41

```
frontmatter    : separator nl blank_line* comment_string? yaml separator nl
```

Description: question-base.md:32-34 says the first paragraph of comments MUST be read as the comment string. `comment_string` is optional and `yaml` is a YAML document, which may itself start with comments and blank lines. With `yaml` stubbed as any sequence of lines other than `---` (/tmp/grammar-audit/frontmatter.lark), `'---\n# c1\n# c2\nid: a\n---\n'` and `'---\n\n# c1\nid: a\n---\n'` each have two parses: one with the comments in `comment_string`, one with no `comment_string` and the comments in `yaml`.

Impact: The grammar alone does not decide whether a comment string exists. The prose resolves it, so the effect is small.

Suggested fix: Add a note under the snippet that `comment_string` is greedy, or define `yaml` so it cannot start with a `COMMENT_LINE`.

F25: Numeric fraction rule accepts a zero denominator

Location: docs/question-types/numeric.md:75

```
fraction     : INTEGER "/" INTEGER
```

Description: numeric.md:106-107 and the `malformed-numeric-answer` rule say a fraction's denominator MUST NOT be zero. The grammar accepts it. Test (/tmp/grammar-audit/test_numeric.py): `'[numeric]: 1/0'` accept, `'[numeric]: 0/0'` accept. The schema pattern (schema/mdq.schema.json:483) uses `[1-9][0-9]*` for the denominator and rejects both.

Impact: The grammar and the schema disagree on `1/0`. The lint rule catches it, so the effect is small.

Suggested fix: Use `fraction : INTEGER "/" POSITIVE_INTEGER` with `POSITIVE_INTEGER : /[1-9][0-9]*/`, as the schema does.

F24: Observations rule is ambiguous on blank lines after the first block

Location: docs/question-types/ordering.md:150-151

```
observations : feedback nl* comment? nl*
```

Description: When `comment` is absent, the two `nl*` are adjacent and the blank lines can go to either. Test (lexer=dynamic, /tmp/grammar-audit/amb.py): `'[ordering]\n- a\n- b\n## [accept]\n> ok\n\n\n- b\n- a\n'` gives an `_ambig` node with two `observations` trees. The parses do not differ in meaning.

Impact: None on the result. A parser generator that rejects ambiguous grammars rejects this rule.

Suggested fix: Use `feedback (nl* comment)? nl*` and `comment (nl* feedback)? nl*`.

Schema mismatches

Severity	Location	Description
LOW	schema/numeric.yaml:87-89	**F13** Schema defines relative tolerance as a division, the spec as a product
LOW	schema/numeric.yaml:48-50	**F14** Schema describes `decimalPlaces` as a display format, the spec as comparison precision
LOW	schema/numeric.yaml:4-7, schema/numeric.yaml:43	**F36** Schema descriptions omit `unit`, `domain`, `decimalPlaces` and domain inference
LOW	schema/ordering.yaml:130-136, docs/question-types/ordering.md:357	**F43** Schema requires a non-empty `extra`, the spec has no such rule

F13: Schema defines relative tolerance as a division, the spec as a product

Location: schema/numeric.yaml:87-89

```
i.e. |submitted - answer| / |answer| <= relative.
```

Description: numeric.md:149-150 defines `|answer - response| ≤ |answer| × relative`. The schema divides by `|answer|`, which is undefined for a zero answer, while the spec form accepts only an exact 0 (numeric.md:217-218).

Impact: An implementer following the schema divides by zero.

Suggested fix: Write the schema description as `|submitted - answer| <= |answer| * relative`.

F14: Schema describes `decimalPlaces` as a display format, the spec as comparison precision

Location: schema/numeric.yaml:48-50

```
The number of decimal places used in the decimal value representation.
```

Description: numeric.md:158 defines `decimalPlaces` as "the precision of the comparison": both sides are rounded to it before the tolerance test. The schema description reads as a formatting hint.

Impact: Tools that rely on the schema description show it as a display setting.

Suggested fix: Use the spec wording: the number of decimal places both the response and the answer are rounded to before the tolerance test.

F36: Schema descriptions omit `unit`, `domain`, `decimalPlaces` and domain inference

Location: schema/numeric.yaml:4-7, schema/numeric.yaml:43

```
description: The domain/kind of number for the response.
```

Description: The top description says the schema extends the base "with `answer` and `tolerance` fields", omitting `unit`, `domain` and `decimalPlaces`. The `domain` description does not say that the domain is inferred when absent (numeric.md:114-117), that a parsed document always carries it (numeric.md:130-131), or that it decides which responses are malformed.

Impact: Tools and readers that rely on the schema get an incomplete model of the field.

Suggested fix: List every field in the top description and describe `domain` with the spec wording.

F43: Schema requires a non-empty `extra`, the spec has no such rule

Location: schema/ordering.yaml:130-136, docs/question-types/ordering.md:357

```
  Lines:
    type: array
    title: Lines
    description: A sequence of lines, in the order they are meant to be read.
    items:
      "$ref": "#/$defs/Line"
    minItems: 1
```

Description: `extra` uses `Lines`, which has `minItems: 1`. The Additional Rules table states the one-line minimum for `accept` and `reject` only. `mdq validate` accepts a `## [extra]` with an empty code block, but rejects an empty `## [accept]` code block with `schema-error` (`accept/0/lines`).

Impact: An empty `## [extra]` and an empty `## [accept]` are handled differently, and the spec explains neither the rule for `extra` nor what happens to an empty section in Markdown.

Suggested fix: Add an Additional Rules row for `extra` and say whether an empty section is dropped or is an error, for every section.

Prioritized recommendations

P1: Fix the numeric grading rules that make implementations disagree on real responses: one rule for response units, exact arithmetic for the tolerance test, and the valid response forms per domain. (Findings: F1, F30, F8, F12, F28)

P1: Rewrite "the same lines" as "the same sequence of lines" in ordering.md, its rules table and lint-codes.md, and say which matching section's feedback is shown. (Findings: F3, F39)

P1: Fix the ordering grammar (indented tag, blank lines, single extra, epilogue) and the section order in question-base.md. (Findings: F18, F20, F21, F24, F2)

P2: Define how ordering lines get their text and level, and tie alternatives and responses to the line pool. (Findings: F4, F42, F44, F5, F46)

P2: Document the body-to-AST mapping of both types, and specify inference for YAML/JSON documents and for a declared decimal domain with no written decimals. (Findings: F31, F32, F16, F40, F43, F9, F29)

P2: Align `unmatched` defaults across types, settle `content`/`highlight` rules, and add an ordering grading corpus. (Findings: F37, F7, F41, F6, F45, F38)

P3: Clean up the numeric lint rules and their rationales, align schema descriptions and example text, fix the shared grammar tokens, and settle presentation details of ordering. (Findings: F10, F11, F34, F35, F13, F14, F36, F15, F33, F22, F23, F25, F26, F17, F47)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Numeric: one rule for response units

**Labels:** high, medium

## Problem

responses.md lets implementations convert a response unit, then calls any other unit malformed,
 and says an incompatible unit is both rejected and graded zero. numeric.md also drops a
frontmatter `unit` that disagrees with the tag unit with no diagnostic.

Options: (A, recommended) same unit is valid, a convertible unit is converted before grading,
any other unit is a `ResponseError`; plus a warning when frontmatter and tag units differ. (B)
only the declared unit is valid, and unit conversion is removed from the spec.

## Evidence

- `docs/responses.md:129-135`: A response unit other than the declared one is both convertible
and malformed

  ```
  Implementations MAY do unit conversions and rewrite the response to a canonical
  form, but they MUST reject a response with a unit that is not compatible with
  the question's unit with grade zero.

  A value the question cannot represent is a malformed response, never a wrong
  answer: a decimal where `domain` is `integer`, or a unit other than the
  declared one.
  ```
- `docs/question-types/numeric.md:22`: A frontmatter `unit` that disagrees with the tag unit is
 silently dropped

  ```
  | unit          | string    | Unit of the response, if given                  |
  ```

## Impact

- **A response unit other than the declared one is both convertible and malformed:** For
`[numeric(g)]: 1000` and the response `{value: 1, unit: kg}`, one implementation scores 1,
another raises `ResponseError`. For `{value: 1, unit: s}`, one scores 0 and another raises an
error.
- **A frontmatter `unit` that disagrees with the tag unit is silently dropped:** A question
whose body says grams is graded in kilograms, with no warning; numeric and short-answer treat
collisions differently.

## Suggested fix

- **A response unit other than the declared one is both convertible and malformed:** State one
rule: a unit equal to the declared one is valid; a different unit that the implementation can
convert is converted before grading; any other unit is a malformed response (`ResponseError`),
not a zero. Remove "with grade zero" and "or a unit other than the declared one".
- **A frontmatter `unit` that disagrees with the tag unit is silently dropped:** Specify the
collision in numeric.md and report it, at least as a warning, when the two units differ.

## Acceptance criteria

- [ ] responses.md states one outcome for each of: same unit, convertible unit, incompatible
unit, unit on a unitless question
- [ ] "with grade zero" no longer appears next to "reject"
- [ ] A lint code reports a frontmatter `unit` that differs from the tag unit, with a corpus
example

--- END ISSUE 1 ---
````

````
--- ISSUE 2 ---
# [Spec] Numeric: specify which responses each domain accepts

**Labels:** medium

## Problem

numeric.md never says what the domain does to responses; responses.md covers part of it and
leaves out decimal responses to fraction questions, unreduced fractions, integer strings and
unitless responses. The response table lists only two of the four forms. Recommended: add a
"Responses" subsection to numeric.md with the grammar per domain, reusing `sign? value`, and
fix the table and the garbled sentence in responses.md.

## Evidence

- `docs/responses.md:102-111`: Numeric response forms are underspecified

  ```
  A string carries an exact rational, which no float can represent. It MUST be
  used when the question's `domain` is `fraction` and the value is not exactly
  representable as a decimal:
  ```
- `docs/question-types/numeric.md:112-117`: The spec never states what the domain does to
responses

  ```
  If not given in the frontmatter, the domain is inferred from how the value and
  the absolute tolerance are written
  ```

## Impact

- **Numeric response forms are underspecified:** Implementations disagree on which responses
are malformed and which are graded.
- **The spec never states what the domain does to responses:** A reader of numeric.md cannot
tell that `[numeric]: 42 +- 5%` rejects the response `43.5` as malformed even though it is
inside the tolerance.

## Suggested fix

- **Numeric response forms are underspecified:** Specify the response grammar per domain (reuse
 `sign? value`), the treatment of unreduced fractions and unitless responses, and update the
table at line 30.
- **The spec never states what the domain does to responses:** Add to "Number type/domain" the
set of responses each domain accepts, and link the malformed-response rule of responses.md.

## Acceptance criteria

- [ ] numeric.md lists the valid response forms for `integer`, `fraction` and `decimal`
- [ ] responses.md table lists number, rational string, decimal string and the `{value, unit}`
struct
- [ ] examples/grading/numeric.yaml covers a decimal response to a fraction question and an
unreduced fraction

--- END ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Spec] Numeric: require exact arithmetic for the tolerance test

**Labels:** high

## Problem

Only the rounding step must use decimal arithmetic. In binary floating point, `3.24` is
rejected by `[numeric]: 3.14 +- 0.1`. Recommended: require the whole comparison in exact
decimal or rational arithmetic, and read a JSON number as its shortest decimal form.

## Evidence

- `docs/question-types/numeric.md:148-154`: Tolerance comparison arithmetic is not specified

  ```
  * Absolute tolerance: The response is accepted if `|answer - response| ≤ abstol`.
  ```

## Impact

Responses on the tolerance boundary grade differently across implementations and languages.

## Suggested fix

Require the whole tolerance test to run in exact decimal/rational arithmetic, with a JSON
number read as its shortest decimal representation.

## Acceptance criteria

- [ ] numeric.md requires exact arithmetic for subtraction, multiplication and comparison
- [ ] A grading example accepts a response exactly on the absolute and on the relative
tolerance boundary

--- END ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Spec] Numeric: fix domain and decimalPlaces inference edge cases

**Labels:** medium

## Problem

Inference reads how numbers are written, which YAML/JSON documents do not preserve, and a
declared `decimal` domain over an integer or fraction value infers `decimalPlaces: 0`, so the
question accepts a whole unit of error. Options: (A, recommended) infer from values for
YAML/JSON documents and add a lint when the inferred `decimalPlaces` is 0 under a declared
`decimal`; (B) require `domain` and `decimalPlaces` in YAML/JSON documents.

## Evidence

- `docs/question-types/numeric.md:166-169`: Inferred `decimalPlaces` of 0 turns a declared
`decimal` domain into rounding to integers

  ```
  When the frontmatter does not give `decimalPlaces` and the domain is
  `decimal`, it is inferred from the body: the larger of the number of decimal
  places written in the value and in the absolute tolerance.
  ```
- `docs/question-types/numeric.md:114-115, docs/question-types/numeric.md:166-168,
docs/question-types/numeric.md:101-102`: Domain and decimal-place inference from a JSON/YAML
number is undefined

  ```
  If not given in the frontmatter, the domain is inferred from how the value and
  the absolute tolerance are written
  ```

## Impact

- **Inferred `decimalPlaces` of 0 turns a declared `decimal` domain into rounding to
integers:** A question meant to be strict silently accepts a wide range.
- **Domain and decimal-place inference from a JSON/YAML number is undefined:** The same
question grades differently depending on whether it was written in Markdown or as data, and
across YAML/JSON libraries.

## Suggested fix

- **Inferred `decimalPlaces` of 0 turns a declared `decimal` domain into rounding to
integers:** Specify the result for a value with no written decimal places (for a fraction, an
exact comparison or a lint), or add a lint when the inferred `decimalPlaces` is 0 under a
declared `decimal` domain.
- **Domain and decimal-place inference from a JSON/YAML number is undefined:** State that for
data documents inference reads values, not spelling (a whole number is an integer, decimal
places of the shortest representation), or require `domain` and `decimalPlaces` in data
documents.

## Acceptance criteria

- [ ] numeric.md states how inference works for a document written as YAML/JSON
- [ ] numeric.md states the result for `domain: decimal` with `[numeric]: 42` and `[numeric]:
1/3`
- [ ] Corpus examples cover both cases

--- END ISSUE 4 ---
````

````
--- ISSUE 5 ---
# [Spec] Numeric: document the body-to-AST mapping and number representation

**Labels:** medium, low

## Problem

numeric.md never declares `answer` and `tolerance` nor how the body maps to them (`+- 5%`
becomes `relative: 0.05`). The absolute tolerance has no string form, so it loses written
digits, and the 32-bit integer limit is unexplained. Recommended: add an "AST" table for the
body, allow a string `tolerance.absolute` with the same grammar as `answer`, and justify or
relax the 32-bit rule.

## Evidence

- `docs/question-types/numeric.md:145-150`: Mapping from the body to `answer` and `tolerance`
is not specified

  ```
  * Relative tolerance: The response is accepted if `|answer - response| ≤ tol`,
  ```
- `docs/question-types/numeric.md:127-128, docs/question-types/numeric.md:97`: Representation
rules cover the answer but not the absolute tolerance

  ```
  The absolute tolerance is an `INTEGER` or a `DECIMAL`, never a fraction (see
  the grammar above and `tolerance.absolute` in the schema, a number).
  ```
- `docs/question-types/numeric.md:97-99`: Integer coercion limit is 32-bit while the precision
rule allows 53-bit

  ```
  It SHOULD be kept as a string if it is not representable in the 32-bit signed
  integer range.
  ```

## Impact

- **Mapping from the body to `answer` and `tolerance` is not specified:** Implementers must
reverse-engineer the data form from the schema and the corpus; a `relative: 5` written for 5%
is a likely error.
- **Representation rules cover the answer but not the absolute tolerance:** The data form of a
tolerance does not round-trip its written precision, which feeds the inference ambiguity of
data documents.
- **Integer coercion limit is 32-bit while the precision rule allows 53-bit:** Surprising
string answers for values like `390000000000`.

## Suggested fix

- **Mapping from the body to `answer` and `tolerance` is not specified:** Add a section that
maps each body part to its field, with `tolerance.relative = percentage / 100`, and use one
form in the formulas.
- **Representation rules cover the answer but not the absolute tolerance:** Apply the answer
representation rules to `tolerance.absolute` (allow the decimal string form), or state that
tolerances are always floats and why.
- **Integer coercion limit is 32-bit while the precision rule allows 53-bit:** Explain the
32-bit choice (integer type in some hosts), or align it with the float rule.

## Acceptance criteria

- [ ] numeric.md maps value, absolute and relative tolerance to `answer`, `tolerance.absolute`
and `tolerance.relative`
- [ ] The 32-bit rule has a stated reason or matches the float rule
- [ ] Schema and spec agree on the type of `tolerance.absolute`

--- END ISSUE 5 ---
````

````
--- ISSUE 6 ---
# [Spec] Numeric: clean up tolerance and domain lint rules

**Labels:** medium, low

## Problem

`missing-tolerance` claims an exact match although decimals are rounded first; `answer-outside-
domain` and `domain-mismatch` overlap at two severities and disagree on whether the absolute
tolerance counts; `relative-tolerance-over-one` blames a units mistake a Markdown percentage
cannot make; `tolerance: {}` silences `missing-tolerance`. Recommended: one code per condition,
 rewrite the three rationales, and treat an empty tolerance as missing.

## Evidence

- `docs/question-types/numeric.md:226-227, docs/question-types/numeric.md:42-43`: `missing-
tolerance` says only an exact match is accepted, but decimals are rounded first

  ```
  [^5]: An exact comparison of a non-integer response is rarely what the author
  means, since the number of digits the student types decides the outcome.
  ```
- `docs/question-types/numeric.md:207-211`: `answer-outside-domain` and `domain-mismatch`
report the same condition at two severities

  ```
  | answer             | warning  | answer-outside-domain          | must be a whole number
when `domain` is "integer"[^6]              |
  ```
- `docs/question-types/numeric.md:210`: `relative-tolerance-over-one` blames a units mistake
that a Markdown percentage cannot make

  ```
  | tolerance.relative | info     | relative-tolerance-over-one    | is a fraction, so a value
above 1 is likely a mistake[^4]          |
  ```
- `docs/question-types/numeric.md:215`: An empty `tolerance` object is neither forbidden nor
covered by `missing-tolerance`

  ```
  | tolerance          | info     | missing-tolerance              | should be defined for a
`decimal` or `fraction` answer[^5]         |
  ```

## Impact

- **`missing-tolerance` says only an exact match is accepted, but decimals are rounded first:**
 The lint rationale misleads authors about what the question accepts.
- **`answer-outside-domain` and `domain-mismatch` report the same condition at two
severities:** Duplicate diagnostics at different severities; implementations disagree on
whether the absolute tolerance counts.
- **`relative-tolerance-over-one` blames a units mistake that a Markdown percentage cannot
make:** Misleading message for Markdown documents.
- **An empty `tolerance` object is neither forbidden nor covered by `missing-tolerance`:** An
empty tolerance hides the lint without adding any tolerance.

## Suggested fix

- **`missing-tolerance` says only an exact match is accepted, but decimals are rounded first:**
 Reword the footnote and lint-codes.md: without a tolerance, a decimal response must round to
the answer at `decimalPlaces`; reconsider whether the lint applies to `fraction`.
- **`answer-outside-domain` and `domain-mismatch` report the same condition at two
severities:** Give each condition one code. State in footnote 3 whether the absolute tolerance
takes part, matching lint-codes.md.
- **`relative-tolerance-over-one` blames a units mistake that a Markdown percentage cannot
make:** Reword the rule as "a relative tolerance above 100% is likely a mistake", or restrict
the units-mistake message to data documents.
- **An empty `tolerance` object is neither forbidden nor covered by `missing-tolerance`:**
Require at least one property in the schema (`minProperties: 1`), or state that an empty
tolerance counts as missing.
````

```
## Acceptance criteria

- [ ] `domain: integer` with `answer: 3.5` reports exactly one code
- [ ] Footnotes and lint-codes.md agree on whether the absolute tolerance counts for `domain-
mismatch`
- [ ] `tolerance: {}` reports `missing-tolerance`
- [ ] The rationales of `missing-tolerance` and `relative-tolerance-over-one` match the
behavior

--- END ISSUE 6 ---
```

````
--- ISSUE 7 ---
# [Spec] Numeric: align schema descriptions and example text with the spec

**Labels:** low

## Problem

The schema describes the relative tolerance as a division (undefined at zero) and
`decimalPlaces` as a display setting, and omits fields and domain inference. numeric.md writes
example labels as H1 headings and claims every question declares a tolerance.

## Evidence

- `schema/numeric.yaml:87-89`: Schema defines relative tolerance as a division, the spec as a
product

  ```
  i.e. |submitted - answer| / |answer| <= relative.
  ```
- `schema/numeric.yaml:48-50`: Schema describes `decimalPlaces` as a display format, the spec
as comparison precision

  ```
  The number of decimal places used in the decimal value representation.
  ```
- `schema/numeric.yaml:4-7, schema/numeric.yaml:43`: Schema descriptions omit `unit`, `domain`,
 `decimalPlaces` and domain inference

  ```
  description: The domain/kind of number for the response.
  ```
- `docs/question-types/numeric.md:38-41`: Numeric body examples are written as H1 headings in a
 Markdown block

  ```
  ```md
  # Simple integer
  [numeric]: 42
  ```
- `docs/question-types/numeric.md:5-6`: Introduction says numeric questions declare a
tolerance, which is optional

  ```
  Numeric questions expect a single numeric answer. They declare both the correct
  answer and tolerance.
  ```

## Impact

- **Schema defines relative tolerance as a division, the spec as a product:** An implementer
following the schema divides by zero.
- **Schema describes `decimalPlaces` as a display format, the spec as comparison precision:**
Tools that rely on the schema description show it as a display setting.
- **Schema descriptions omit `unit`, `domain`, `decimalPlaces` and domain inference:** Tools
and readers that rely on the schema get an incomplete model of the field.
- **Numeric body examples are written as H1 headings in a Markdown block:** Minor confusion;
the block looks like a document but is not one.
- **Introduction says numeric questions declare a tolerance, which is optional:** Readers may
think a tolerance is required.

## Suggested fix

- **Schema defines relative tolerance as a division, the spec as a product:** Write the schema
description as `|submitted - answer| <= |answer| * relative`.
- **Schema describes `decimalPlaces` as a display format, the spec as comparison precision:**
Use the spec wording: the number of decimal places both the response and the answer are rounded
 to before the tolerance test.
- **Schema descriptions omit `unit`, `domain`, `decimalPlaces` and domain inference:** List
every field in the top description and describe `domain` with the spec wording.
- **Numeric body examples are written as H1 headings in a Markdown block:** Use a table of
````
````

```
examples or one block per example with the label in prose.
- **Introduction says numeric questions declare a tolerance, which is optional:** Write: "They
declare the correct answer and an optional tolerance."

## Acceptance criteria

- [ ] Schema descriptions of `relative`, `decimalPlaces`, `domain` and the top description
match numeric.md
- [ ] Body examples no longer use `#` lines inside an `md` block
- [ ] The introduction says the tolerance is optional

--- END ISSUE 7 ---
```

````
--- ISSUE 8 ---
# [Spec] Grammar: fix line endings, frontmatter and numeric fraction rules

**Labels:** medium, low

## Problem

`nl` and `COMMENT_LINE` allow CRLF to parse as two line endings; the frontmatter grammar does
not force the first comment paragraph into `comment_string`; the numeric `fraction` accepts a
zero denominator that the schema rejects; schema patterns anchored with `$` accept a trailing
newline in Python but not in JavaScript.

## Evidence

- `docs/references/grammar.md:29, docs/references/grammar.md:63`: nl and COMMENT_LINE can split
 CRLF into two line endings

  ```lark
  nl : /\r\n|\r|\n/
  ```
- `docs/question-types/question-base.md:41`: Frontmatter grammar does not force the first
comment paragraph into comment_string

  ```lark
  frontmatter    : separator nl blank_line* comment_string? yaml separator nl
  ```
- `docs/question-types/numeric.md:75`: Numeric fraction rule accepts a zero denominator

  ```lark
  fraction     : INTEGER "/" INTEGER
  ```
- `docs/references/grammar.md:17-19, schema/mdq.schema.json:396, schema/mdq.schema.json:483,
schema/mdq.schema.json:490`: Schema forms of SLUG, UNIT and the numeric answer accept a
trailing newline in Python

  ```json
  "pattern": "^[a-zA-Z0-9]+(?:[-_][a-zA-Z0-9]+)*$",
  ```

## Impact

- **nl and COMMENT_LINE can split CRLF into two line endings:** A regex engine or a parser that
 backtracks into the alternation can count a CRLF as a line plus a blank line. Blank lines end
the comment string and are significant in several bodies, so the meaning of a Windows file can
change.
- **Frontmatter grammar does not force the first comment paragraph into comment_string:** The
grammar alone does not decide whether a comment string exists. The prose resolves it, so the
effect is small.
- **Numeric fraction rule accepts a zero denominator:** The grammar and the schema disagree on
`1/0`. The lint rule catches it, so the effect is small.
- **Schema forms of SLUG, UNIT and the numeric answer accept a trailing newline in Python:**
The two implementations validate the same YAML or JSON document differently: an id, a unit or
an answer with a trailing line feed (easy to get from a YAML block scalar) passes the Python
validator and fails the TypeScript one.

## Suggested fix

- **nl and COMMENT_LINE can split CRLF into two line endings:** Use `/\r\n|\r(?!\n)|\n/` for
`nl` and `(\r\n|\r(?!\n)|\n)` at the end of `COMMENT_LINE`.
- **Frontmatter grammar does not force the first comment paragraph into comment_string:** Add a
 note under the snippet that `comment_string` is greedy, or define `yaml` so it cannot start
with a `COMMENT_LINE`.
- **Numeric fraction rule accepts a zero denominator:** Use `fraction : INTEGER "/"
POSITIVE_INTEGER` with `POSITIVE_INTEGER : /[1-9][0-9]*/`, as the schema does.
- **Schema forms of SLUG, UNIT and the numeric answer accept a trailing newline in Python:**
End the patterns with `(?![\s\S])` or `$(?![\s\S])` instead of `$`, or have the Python
validator use `re.fullmatch` semantics, and state the anchoring rule in grammar.md.

## Acceptance criteria
````

```
- [ ] `nl` uses `\r(?!\n)` and CRLF has one derivation
- [ ] `fraction` rejects `1/0` in the grammar
- [ ] Schema patterns reject `'Q1\n'`, `'kg\n'` and `'42\n'` in both Python and JavaScript
- [ ] The frontmatter grammar or a note under it makes `comment_string` greedy

--- END ISSUE 8 ---
```

````
--- ISSUE 9 ---
# [Spec] Ordering: make the grammar match the prose and the corpus

**Labels:** high, medium, low, info

## Problem

The ordering grammar rejects an indented tag and a blank line before a section, accepts several
 `## [extra]` sections, is ambiguous on blank lines in observations, and has no epilogue.
question-base.md puts type-specific sections after the epilogue, ordering.md puts them before.
Recommended: rewrite the rule with `ws?`, `(nl* section)*`, one optional extra, an epilogue,
and update question-base.md to place type sections before the epilogue.

## Evidence

- `docs/question-types/ordering.md:141`: Ordering tag rule rejects an indented tag

  ```lark
  ordering : "[" "ordering" "]" nl content section*
  ```
- `docs/question-types/ordering.md:141`: Ordering grammar rejects a blank line between a
content block and the next section

  ```lark
  ordering : "[" "ordering" "]" nl content section*
  ```
- `docs/question-types/ordering.md:144`: Ordering grammar accepts more than one extra section

  ```lark
  section : ordering_extra | ordering_accept | ordering_reject
  ```
- `docs/question-types/ordering.md:150-151`: Observations rule is ambiguous on blank lines
after the first block

  ```lark
  observations : feedback nl* comment? nl*
  ```
- `docs/question-types/ordering.md:181-183`: Ordering puts the epilogue after its sections; the
 base puts special sections after the epilogue

  ```
  The sections below follow the content block directly. The epilogue, if any,
  comes after the last section; prose between the block and a section is an
  error (`parse-error`).
  ```

## Impact

- **Ordering tag rule rejects an indented tag:** The grammar rejects a document that the prose
and the corpus declare valid. An implementation that follows the grammar fails the shared test
suite; one that follows the corpus has no grammar for what indentation it may accept.
- **Ordering grammar rejects a blank line between a content block and the next section:** The
grammar accepts only the layout without a blank line, which is the less common one, and
contradicts the corpus.
- **Ordering grammar accepts more than one extra section:** The grammar accepts a document the
prose forbids. The prose is clear, so the effect is small.
- **Observations rule is ambiguous on blank lines after the first block:** None on the result.
A parser generator that rejects ambiguous grammars rejects this rule.
- **Ordering puts the epilogue after its sections; the base puts special sections after the
epilogue:** An author following the base document writes the epilogue before `## [accept]` and
gets a `parse-error`. Readers cannot tell which document is normative.

## Suggested fix

- **Ordering tag rule rejects an indented tag:** Start `ordering` with `ws?`, as `answer` of
numeric.md does, and do the same for the short-answer and essay rules.
- **Ordering grammar rejects a blank line between a content block and the next section:** Allow
 blank lines before each section: `ordering : "[" "ordering" "]" nl content (nl* section)*`, or
 state in the prose that `md_code_block` and `md_ul` include the blank lines after them.
- **Ordering grammar accepts more than one extra section:** Express the limit in the grammar,
````

````
for example `ordering : ... nl content (accept_or_reject)* (ordering_extra
(accept_or_reject)*)?`, or add a note under the snippet that the limit is not in the grammar.
- **Observations rule is ambiguous on blank lines after the first block:** Use `feedback (nl*
comment)? nl*` and `comment (nl* feedback)? nl*`.
- **Ordering puts the epilogue after its sections; the base puts special sections after the
epilogue:** Make question-base.md describe the type-specific sections as part of the body (or
as coming before the epilogue), and add the epilogue to the ordering grammar.

## Acceptance criteria

- [ ] The rule accepts every valid ordering example in the corpus
- [ ] `## [extra]` twice is rejected by the grammar
- [ ] question-base.md and ordering.md agree on where the epilogue goes

--- END ISSUE 9 ---


--- ISSUE 10 ---
# [Spec] Ordering: define "the same lines" and which matching section wins

**Labels:** high, medium

## Problem

"Hold the same lines" can mean a multiset, which would forbid most questions with accept and
reject sections. When several sections match one response, the spec does not say whose feedback
 is shown. Recommended: say "the same sequence of lines" everywhere, and show the feedback of
the first matching section in document order.

## Evidence

- `docs/question-types/ordering.md:208-212`: "Hold the same lines" can mean the same multiset
or the same sequence

  ```
  A question SHOULD NOT declare two sections whose content
  blocks hold the same lines once **all forms of normalization** have been
  applied
  ```
- `docs/question-types/ordering.md:296-298`: Feedback is undefined when a response matches
several sections

  ```
  A response that matches an accepted or a rejected answer shows that section's
  feedback, if it declares one.
  ```

## Impact

- **"Hold the same lines" can mean the same multiset or the same sequence:** An implementation
that reads it as a multiset raises a critical `accept-reject-overlap` on valid questions.
- **Feedback is undefined when a response matches several sections:** Implementations show
different feedback for the same response.

## Suggested fix

- **"Hold the same lines" can mean the same multiset or the same sequence:** Write "the same
sequence of lines (same lines, same order)" in the prose, in the three Additional Rules rows
and in lint-codes.md.
- **Feedback is undefined when a response matches several sections:** State a precedence rule,
for example: the `[ordering]` block first, then sections in document order; the first match
decides the score and the feedback.

## Acceptance criteria

- [ ] ordering.md, its Additional Rules table and lint-codes.md use "the same sequence of
lines"
- [ ] ordering.md states which feedback is shown when several sections match

--- END ISSUE 10 ---
````

````
--- ISSUE 11 ---
# [Spec] Ordering: constrain alternatives and responses to the line pool

**Labels:** medium, low

## Problem

Nothing ties `## [accept]`/`## [reject]` lines, or response lines, to the lines shown to the
student. A typo makes an alternative unreachable; a reject equal to the answer key has no
outcome; a response with a foreign line or a changed level under `fixed` is neither malformed
nor wrong.

## Evidence

- `docs/question-types/ordering.md:203-212`: No rule ties accepted and rejected orderings to
the lines shown to the student

  ```
  The accept/reject blocks start with a `## [accept]` or `## [reject]` header,
  followed by optional observations and the content block.
  ```
- `docs/responses.md:187-190`: A response that changes a level under `fixed` has no defined
outcome

  ```
  The indentation level is always present. Under `indentation: fixed` it is the
  authored level, which the student cannot change
  ```

## Impact

- **No rule ties accepted and rejected orderings to the lines shown to the student:** A typo in
 an alternative makes it unreachable without any diagnostic. A reject that equals the answer
key has no defined outcome.
- **A response that changes a level under `fixed` has no defined outcome:** One implementation
rejects the response as malformed, another grades it 0 or leaves it pending.

## Suggested fix

- **No rule ties accepted and rejected orderings to the lines shown to the student:** Add
rules: alternative lines must come from the presented pool (lint, warning); a reject must not
equal the answer key (critical).
- **A response that changes a level under `fixed` has no defined outcome:** State that a
response line whose text and level are not in the presented pool, or that is used more often
than provided, is a malformed response (`ResponseError`).

## Acceptance criteria

- [ ] A lint reports an alternative line outside `[ordering]` plus `## [extra]`
- [ ] A reject equal to the answer key after normalization is a critical error
- [ ] responses.md states that a foreign line, a surplus copy, or a changed level under `fixed`
 is a malformed response

--- END ISSUE 11 ---
````

````
--- ISSUE 12 ---
# [Spec] Ordering: define line text, levels and indentation unit

**Labels:** high, medium, low

## Problem

The spec does not define the level of a `ul` line, tab expansion, the level of a blank line,
the step offered under `strict`, or whether list markers and trailing whitespace belong to the
text. Recommended: `text` content is always level 0, tabs expand to the next multiple of 4,
trailing whitespace is stripped, and one step is one unit.

## Evidence

- `docs/question-types/ordering.md:261-266`: Indentation of `ul` lines is not defined

  ```
  The **indentation unit** is inferred from the lines of the question. It
  considers all lines in the `[ordering]`, `## [extra]`, `## [accept]` and
  `## [reject]` blocks, normalizes a tabulation as 4 spaces, and takes the
  greatest common divisor of the leading whitespace of every non-blank line that
  is indented at all.
  ```
- `docs/question-types/ordering.md:261-272`: Indentation step offered to the student under
`strict` is not defined

  ```
  A line's **indentation level** is its leading whitespace divided by that unit,
  and that level -- not the raw whitespace -- is what a response carries and
  what `"strict"` compares.
  ```
- `docs/question-types/ordering.md:165-166`: Whitespace and list markers in line text are not
specified

  ```
  If it is a `ul` list, the question infers `content: "text"`. Each line is
  derived from the respective list item, preserving its markdown source verbatim.
  ```

## Impact

- **Indentation of `ul` lines is not defined:** Two implementations give different `[level,
text]` pairs for the same `ul` question, so the same response grades differently.
- **Indentation step offered to the student under `strict` is not defined:** Under `strict`, a
student who indents a loop body once submits level 1 while the key expects level 2; responses
stored before a section was added stop matching.
- **Whitespace and list markers in line text are not specified:** A line with a trailing space
can be different from the same line without it in one implementation and equal in another.

## Suggested fix

- **Indentation of `ul` lines is not defined:** State that `text` content is always at level 0
(nested lists are a `parse-error`), or define levels from list nesting. Define tab expansion
and the level of a blank line.
- **Indentation step offered to the student under `strict` is not defined:** State that the
interface moves a line by one unit per step and renders levels with the inferred unit, or
derive the unit from the `[ordering]` block alone and lint other sections that do not fit it.
- **Whitespace and list markers in line text are not specified:** State that the text excludes
the marker and its following whitespace, and whether trailing whitespace is stripped for both
content types.

## Acceptance criteria

- [ ] ordering.md defines text and level for code lines and `ul` items
- [ ] Tab expansion and blank-line levels are defined
- [ ] A corpus example covers a nested `ul`

--- END ISSUE 12 ---
````

````
--- ISSUE 13 ---
# [Spec] Ordering: clarify `content`, `highlight` and fence languages

**Labels:** medium, low

## Problem

A declared `content` that disagrees with the body block has no defined effect, the schema's
`default: text` is undocumented, fence languages in accept/reject sections are not addressed,
and essay calls the same field `input`. Recommended: make a mismatch a `parse-error`, document
the default, ignore fence languages in every section, and pick one name for the field across
essay and ordering.

## Evidence

- `docs/question-types/ordering.md:106-108`: Declared `content` that disagrees with the body
block has no meaning

  ```
  [^1]: `content = "code" | "text"`. Both fields are OPTIONAL, and when given
  they override what the body implies.
  ```
- `docs/question-types/ordering.md:193-194`: Language of `## [accept]` and `## [reject]` fences
 is not addressed

  ```
  The highlight of fenced code blocks is ignored.
  ```
- `docs/question-types/ordering.md:100-101, docs/question-types/essay.md:21-22`: Ordering
`content` and essay `input` declare the same code/text choice under different names

  ```
  | content        | string          | Either "code" or "text". Inferred from the body when
omitted[^1].          |
  ```

## Impact

- **Declared `content` that disagrees with the body block has no meaning:** Implementations can
 accept, reject, or reinterpret the same document.
- **Language of `## [accept]` and `## [reject]` fences is not addressed:** A different fence
language in an alternative can be accepted, warned or rejected depending on the implementation.
- **Ordering `content` and essay `input` declare the same code/text choice under different
names:** Authors must remember two names for one concept, and a code ordering question without
a language gets no notice while an essay does.

## Suggested fix

- **Declared `content` that disagrees with the body block has no meaning:** Make a mismatch
between declared `content` and the body block a `parse-error` (or a warning with a defined
effect). Document the `text` default for documents written as YAML/JSON.
- **Language of `## [accept]` and `## [reject]` fences is not addressed:** Move the sentence to
 the Body section and make it apply to every section after `[ordering]`.
- **Ordering `content` and essay `input` declare the same code/text choice under different
names:** Use one field name for both types, or document the difference; add the missing-
highlight info lint to ordering or remove it from essay.

## Acceptance criteria

- [ ] `content: text` over a code block has a defined outcome with a corpus example
- [ ] The prose documents the `text` default
- [ ] Ordering and essay use the same field name for the code/text choice

--- END ISSUE 13 ---
````

````
--- ISSUE 14 ---
# [Spec] Ordering: align `unmatched`, manual scores and grading examples

**Labels:** medium

## Problem

Ordering defaults `unmatched` to `manual` although it always has an answer key, while short-
answer defaults to `incorrect` in that case. The binary score rule does not say whether it
binds a manual score. No grading corpus exists for ordering. Recommended: default to
`incorrect` like short-answer, limit the binary rule to automatic grading, and add
examples/grading/ordering.yaml.

## Evidence

- `docs/question-types/ordering.md:118-120, docs/question-types/short-answer.md:266-267`:
`unmatched` defaults differ between ordering and short-answer

  ```
  [^4]: `Unmatched = "manual" | "incorrect"`. `"manual"` (the default) leaves a
  ```
- `docs/question-types/ordering.md:326-332`: Ordering score is "always binary" but a pending
response is scored by hand

  ```
  The score is always binary -- `1` or `0`, with no partial credit for a nearly
  correct ordering.
  ```
- `docs/question-types/ordering.md:308-332`: No grading examples for ordering in the corpus

  ```
  Grading compares the response to the answer keys **by content**: lines carry no
  ```

## Impact

- **`unmatched` defaults differ between ordering and short-answer:** An author who learns
`unmatched` on short-answer expects unmatched ordering responses to score 0 by default; instead
 they stay pending and need manual grading.
- **Ordering score is "always binary" but a pending response is scored by hand:** A host may
refuse a partial manual score that another host accepts.
- **No grading examples for ordering in the corpus:** The grading rules of ordering, the most
complex among the automatically graded types, are not checked by the shared test suite, so the
Python and TypeScript implementations can diverge.

## Suggested fix

- **`unmatched` defaults differ between ordering and short-answer:** Use one default rule for
`unmatched` across types, or state in ordering.md why its default differs from short-answer.
- **Ordering score is "always binary" but a pending response is scored by hand:** State that
the binary rule applies to automatic grading only, and that a manual score is any value in [0,
1] (or restrict it explicitly).
- **No grading examples for ordering in the corpus:** Add examples/grading/ordering.yaml
covering each normalization, each indentation mode, duplicates, distractors, and both
`unmatched` values.

## Acceptance criteria

- [ ] Ordering and short-answer state the same default rule for `unmatched`
- [ ] ordering.md says what range a manual score may take
- [ ] examples/grading/ordering.yaml covers dedent, skip-blanks, duplicates, distractors and
unmatched

--- END ISSUE 14 ---
````

````
--- ISSUE 15 ---
# [Spec] Ordering: document the body-to-AST mapping

**Labels:** medium, low

## Problem

The prose never says the blocks become `lines`, `extra`, `accept` and `reject`, nor how `>` and
 `!` blocks become `feedback` and `comment`. The schema requires a non-empty `extra`, which the
 rules table does not state.

## Evidence

- `docs/question-types/ordering.md:133-136, docs/question-types/ordering.md:354-358`: Mapping
from the ordering body to `lines`, `extra`, `accept` and `reject` is not specified

  ```
  | lines                 | critical | schema-error                | must have at least two
lines (schema)                              |
  ```
- `schema/ordering.yaml:130-136, docs/question-types/ordering.md:357`: Schema requires a non-
empty `extra`, the spec has no such rule

  ```
    Lines:
      type: array
      title: Lines
      description: A sequence of lines, in the order they are meant to be read.
      items:
        "$ref": "#/$defs/Line"
      minItems: 1
  ```

## Impact

- **Mapping from the ordering body to `lines`, `extra`, `accept` and `reject` is not
specified:** Readers must reverse-engineer the YAML/JSON form from the schema and the corpus;
the feedback and comment text can be serialized differently by two implementations.
- **Schema requires a non-empty `extra`, the spec has no such rule:** An empty `## [extra]` and
 an empty `## [accept]` are handled differently, and the spec explains neither the rule for
`extra` nor what happens to an empty section in Markdown.

## Suggested fix

- **Mapping from the ordering body to `lines`, `extra`, `accept` and `reject` is not
specified:** Add a short paragraph or a YAML example that shows the parsed form of the full
example, and state how feedback and comment text is extracted.
- **Schema requires a non-empty `extra`, the spec has no such rule:** Add an Additional Rules
row for `extra` and say whether an empty section is dropped or is an error, for every section.

## Acceptance criteria

- [ ] ordering.md maps each block to its field and describes the alternative object
- [ ] The Additional Rules table lists the `extra` minimum, or the schema drops it

--- END ISSUE 15 ---
````

````
--- ISSUE 16 ---
# [Spec] Ordering: settle presentation details

**Labels:** low

## Problem

Indented blockquotes in observations are left to the implementation, and "never shown that
order" conflicts with a random shuffle that can reproduce the key.

## Evidence

- `docs/question-types/ordering.md:234-237`: Indented blockquotes in observations are
implementation-defined

  ```
  It is up to the implementation to decide if it should ignore the indentation,
  issue a warning or even an error.
  ```
- `docs/question-types/ordering.md:7-9`: "Never shown that order" conflicts with a random
shuffle

  ```
  Students are never shown that order --
  implementations MUST present the lines shuffled, in a random or pseudo-random
  arrangement
  ```

## Impact

- **Indented blockquotes in observations are implementation-defined:** Portability of documents
 between implementations.
- **"Never shown that order" conflicts with a random shuffle:** An implementation that follows
"random" literally can show the solution.

## Suggested fix

- **Indented blockquotes in observations are implementation-defined:** Fix one behavior: accept
 up to 3 spaces of indentation, as CommonMark does, and lint it.
- **"Never shown that order" conflicts with a random shuffle:** Say that implementations MUST
NOT present the answer key's order, and reshuffle when the permutation equals it.

## Acceptance criteria

- [ ] ordering.md fixes one behavior for indented `>` and `!` lines
- [ ] ordering.md says whether a shuffle equal to the answer key must be redone

--- END ISSUE 16 ---
````